FeeFlow AI

Technical Design Document & Take-Home Assignment Submission

Isocode Labs ? Software Engineering Intern

GitHub repository URL: NOT AVAILABLE IN THIS CHECKOUT (BLOCKING SUBMISSION ITEM)

Live application URL: Not deployed or verified

Evidence date: 8 November 2026

This document describes the implementation actually present in the repository. Features marked as assumptions, partial, unavailable, or unverified are not represented as production capabilities.

1. Project overview

FeeFlow AI is a coaching-institute fee-management demonstration. Its core build provides a server-authoritative fee ledger, parent portal, mock payment lifecycle, automated reminder state machine, pro-rata refund review, Accounts dashboard, and a human-reviewed student-support risk workflow.

2. Problem statement and target users

The system provides one source of truth for fee plans, instalment balances, payment history, due-date reminders, refund calculations, and operational follow-up. Target roles are Parent, Accounts Executive, and Counsellor. The Counsellor role is limited to human review of risk tasks.

3. Functional requirements and compliance

Requirement	Status	Evidence / limitation
Fee ledger and instalment balances	Implemented and verified	Integer-paise ledger, server-derived totals, payment history, and tests.
Parent OTP login	Implemented but not externally verified	OTP expiry, HMAC, attempts, and atomic consumption tested; SMTP delivery unverified.
Payment links and parent history	Implemented and verified locally	Parent-owned mock order, success/failure, settlement, and receipt tests.
Reminder automation/cancellation	Implemented and verified locally	Idempotent schedules, mock delivery state, and cancellation after settlement.
Refund calculator/review	Implemented and verified locally	Ledger-backed policy snapshot, approval/rejection, and validation tests.
Centre/batch dashboard	Implemented and verified locally	Collection, ageing, trends, CSV export, and browser dashboard tests.
AI dropout-risk task	Implemented and verified locally	Rules baseline creates counsellor task only; no fee/message action.
False-positive handling	Implemented and verified locally	Counsellor can record valid or false-flag feedback once.
Hindi/English support	Partially implemented	Reminder language preference exists; full UI localization is absent.
GST receipts	Partially implemented	Configurable GST note exists; tax calculation is not implemented.
DPDP parental consent	Not implemented	No consent record or workflow exists.
Tally export	Not implemented	Accounts CSV exists; Tally-specific export does not.
Attendance/test-score upload	Partially implemented	Stored fields feed risk logic; upload workflow is absent.
Unmatched-payment queue	Not implemented	No unmatched-payment queue or UI exists.

4. Explicit assumptions

• All demo identities and student records are fictional.

• Amounts are integer paise; no floating-point financial arithmetic is used.

• The refund policy is a demonstration assumption, not legal or institutional policy.

• Mock payment and reminder providers contact no external customer or provider.

• Risk scores are explainable support signals, not dropout probabilities or adverse decisions.

• Production requires transaction-capable MongoDB, managed SMTP, secrets management, HTTPS, monitoring, and provider certification.

5. Technology stack and justification

Layer	Technology	Reason
Client	React, Vite, React Router, Axios, Recharts	Responsive role-based portal and dashboard charts.
API	Node.js, Express, Zod, Helmet, rate limiting	Typed validation boundary, middleware security, and service composition.
Persistence	MongoDB/Mongoose	Document models for students, embedded instalments, workflows, and audit state.
Authentication	bcryptjs, HMAC OTP, JWT	Password hashing, one-time verification, and stateless access tokens.
Risk service	FastAPI, Pydantic, Pandas	Private, locally deployable, explainable rules endpoint.
Tests	Node test runner, pytest, Playwright, Vite build	Backend, AI, browser, and production-build verification.

6. System architecture

The React browser calls the authenticated Express API with bearer JWTs. Express validates requests, applies role checks, performs financial calculations, and persists MongoDB records. Accounts-only risk assessment calls the private FastAPI service; the API validates the response before storing an assessment and opening a counsellor task.

Architecture diagram: Browser (Parent / Accounts / Counsellor) ? Express API (auth, validation, ledger, payments, reminders, refunds, risk) ? MongoDB; Express API ? private FastAPI rules service. The offline scikit-learn experiment is synthetic-only and is not used by predictions.

7. Database schema and relationships

Model	Purpose
User	Normalized email, role, linked student, active state, password hash.
Student	Fictional profile, parent contact, attendance, test history, payment promises.
FeePlan	Gross/discount/net amounts and embedded instalments with payment references.
Payment / MockPaymentOrder	Completed history and short-lived parent-bound order state.
Reminder	Idempotent scheduled message state, retries, cancellation and audit metadata.
Refund	Versioned policy inputs, calculation, review, provider processing state.
CounsellorTask / RiskAssessment	Risk explanation, feature snapshot, review feedback and assessment history.
Invitation / OtpChallenge	Hashed one-time onboarding/login verification state.

8. Fee ledger and instalment rules

Net fee equals gross fee minus discount. A plan contains three or four instalments whose amounts sum to net fee; remainder paise are distributed deterministically. Outstanding balance is amount minus paid amount and overpayment is rejected. Overdue status uses whole UTC calendar days after due date. Totals are derived server-side from persisted instalments.

9. Payment lifecycle

A parent can create an order only for the linked student. The server calculates the outstanding amount, creates a short-lived order, and settlement rechecks ownership, expiry, signed callback, current balance, and transaction uniqueness. Successful settlement updates the ledger, payment history, order, and reminders transactionally. Mock settlement is disabled in production. Razorpay primitives exist but live provider certification is not verified.

10. Reminder automation and cancellation

Default offsets are 3 days before, on due date, and 3, 7, and 15 days after. Idempotency keys prevent duplicate schedules. Processing rechecks payment state, refreshes partial balances, stops fully paid reminders, and records delivery/retry/failure/cancellation metadata. The current provider is mock-only and sends no SMS.

11. Pro-rata refund calculation

Refundable amount = max(0, amount paid ? fee earned ? non-refundable charges). Fee earned is net course fee multiplied by completed whole UTC days divided by course duration, rounded to the nearest paise. Accounts users approve or reject a persisted calculation; approval does not automatically disburse money. Razorpay refund processing claims a record atomically as processing and sends a provider idempotency key. Failed calls remain processing for reconciliation.

12. AI dropout-risk methodology and safeguards

The API derives missed instalments, overdue days, attendance, latest test-score trend, and broken payment promises. FastAPI applies bounded explainable rules and returns a score, level, reasons, feature breakdown, missing features, and limitations. Medium/high results create or update a counsellor task only. The flag cannot change fees, payments, reminders, enrolment, penalties, or parent messages. Counsellors can record valid/false-flag feedback. No labelled historical dataset is present; the synthetic ML experiment is not the prediction model.

13. Security and role access

• bcrypt password hashes and active-user checks on every JWT request.

• HMAC OTP digests with expiry, attempt limits, and atomic consumption.

• Hashed invitation tokens with expiry and one-time use.

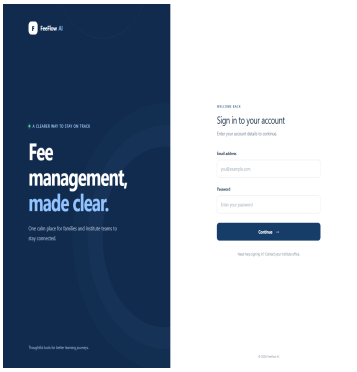
• Strict Zod/Pydantic input validation, Helmet, exact-origin CORS, body-size limits, and authentication rate limiting.

• Parent ownership checks for fee, payment, and receipt routes; Accounts-only financial administration; Counsellor-only risk feedback.

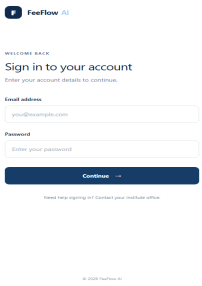
• Request logs redact secrets, tokens, authorization, cookies, OTPs, and API keys.

14. UI screenshots and design decisions

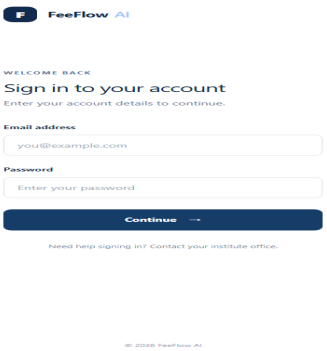
The repository contains genuine login screenshots at desktop, tablet, and mobile sizes under docs/screenshots/. The UI uses role-specific navigation, responsive cards/tables, status labels, loading/error/empty states, and mobile-first payment actions. Authenticated workflows were verified using Playwright synthetic API fixtures; no screenshot is presented as a live production deployment.



login-desktop.png



login-tablet.png



login-mobile.png

15. Testing strategy and actual results

Suite	Actual result
Backend	85 tests, 83 passed, 0 failed, 2 documented skips
AI service	7 passed
Focused payment/refund/provider tests	6 passed
Frontend production build	Passed
Playwright	6 passed
JavaScript/Python syntax	Passed
git diff --check	Passed
npm production audit	0 high/critical vulnerabilities reported

16. Known limitations and deliberately excluded features

No verified public GitHub URL or live deployment URL exists in this checkout. Razorpay sandbox, managed SMTP, AWS, production TLS, monitoring, private AI networking, MongoDB backup/restore, DPDP consent, Tally export, unmatched-payment queue, full localization, and attendance/test-score upload are not verified or not implemented as identified in the compliance table.

17. Future improvements

• Add verified repository and deployment links.

• Add provider-backed payment, webhook, refund, and outcome reconciliation tests.

• Implement unmatched-payment Accounts queue and Tally export.

• Add consent records, retention/deletion controls, and institution-approved privacy workflows.

• Add governed attendance/test-score import and complete Hindi/English localization.

• Complete staging acceptance for SMTP, TLS, backups, monitoring, and private AI networking.

18. Submission and production status

Submission artifact: this TDD PDF plus deployment.md are packaged separately as feeflow-ai-isocode-submission.zip. Submission readiness is assessed separately from production readiness. The application remains NOT READY FOR PRODUCTION because mandatory external acceptance evidence is absent.